

SKILL 7:

Process Synchronization and Child Process Monitoring Using Waitpid()

```
nirvighna@DESKTOP-VS57D2  x  +  v  -  □  ×

#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main(void)
{
    pid_t pid;
    int status;

    printf("Parent process started. PID = %d\n", getpid());

    pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (pid == 0)
    {
        printf("Child process started. PID = %d, PPID = %d\n",
               getpid(), getppid());

        printf("Child is working...\n");
        sleep(3);

        printf("Child process exiting.\n");
        return 42;
    }
    else
    {
        printf("Parent is waiting for child PID %d...\n", pid);

        if (waitpid(pid, &status, 0) == -1)
        {
            perror("waitpid");
            return 1;
        }

        printf("Parent process exiting.\n");
        return 0;
    }
}

EOF nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ gcc -Wall -Wextra -g src/waitpid_demo.c -o bin/waitpid_demo
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ls -l bin/waitpid_demo
-rwxr-xr-x 1 nirvighna nirvighna 18080 Sep 27 15:48 bin/waitpid_demo
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/waitpid_demo
Parent process started. PID = 513
Parent is waiting for child PID 514...
Child process started. PID = 514, PPID = 513
Child is working...
Child process exiting.
Child exited normally with status 42.
Parent process exiting.
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ echo $PATH
/home/nirvighna/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/Program Files/Common Files/Oracle/Java/javapath:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/java8path:/mnt/c/Program Files (x86)/Common Files/Oracle/Java/javapath:/mnt/c/WINDOWS/system32:/mnt/c/WINDOWS:/mnt/c/WINDOWS/System32/Wbem:/mnt/c/WINDOWS/System32/WindowsPowerShell/v1.0:/mnt/c/WINDOWS/System32/OpenSSH:/mnt/c/MinGW/bin:/mnt/c/Program Files/nodejs:/mnt/c/Users/LENOVO/AppData/Local/Programs/Python/Launcher:/mnt/c/Users/LENOVO/.local/bin:/mnt/c/Users/LENOVO/AppData/Local/Microsoft/WindowsApps:/mnt/c/Users/LENOVO/AppData/Local/Programs/Microsoft VS Code/bin:/mnt/d/SECTION 10 C-PROGRAMME/bin:/mnt/c/Users/LENOVO/AppData/Local/GitHubDesktop/bin:/mnt/c/Users/LENOVO/AppData/Roaming/npm:/mnt/c/Users/LENOVO/AppData/Local/Python/bin:/snap/bin
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
```

TASK 2:

```
nirvighna@DESKTOP-VS57D2  ×  +  ▾  -  □  ×

#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <limits.h>

int main(int argc, char *argv[])
{
    char *path;
    char *path_copy;
    char *dir;
    char full_path[PATH_MAX];

    if (argc != 2)
    {
        printf("Usage: %s <command>\n", argv[0]);
        return 1;
    }

    path = getenv("PATH");

    if (path == NULL)
    {
        fprintf(stderr, "PATH variable is not set.\n");
        return 1;
    }

    path_copy = malloc(strlen(path) + 1);

    if (path_copy == NULL)
    {
        perror("malloc");
        return 1;
    }

    strcpy(path_copy, path);

    dir = strtok(path_copy, ":");

    while (dir != NULL)
    {
        if (access(dir, F_OK) == 0)
        {
            printf("Executable found: %s\n", dir);
            return 0;
        }
        dir = strtok(NULL, ":");
    }
    printf("EOF\n");
    return 1;
}

nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ls -l bin/path_search
-rwxr-xr-x 1 nirvighna nirvighna 19576 Sep 27 15:55 bin/path_search
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/path_search ls
Executable found: /usr/bin/ls
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ls -l bin
total 124
-rwxr-xr-x 1 nirvighna nirvighna 18080 Sep 25 17:22 file_copy
-rwxr-xr-x 1 nirvighna nirvighna 19576 Sep 27 15:55 path_search
-rwxr-xr-x 1 nirvighna nirvighna 19784 Sep 21 05:34 practical1
-rwxr-xr-x 1 nirvighna nirvighna 18056 Sep 25 07:42 process_states
-rwxr-xr-x 1 nirvighna nirvighna 20560 Sep 27 12:34 session6_shell
-rwxr-xr-x 1 nirvighna nirvighna 18080 Sep 27 15:48 waitpid_demo
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
```

SKILL 8:

To Detect Variable References, Expand Values, Handle Undefined Variables Update Tokens, Support Nested Variables, Test Expansion Logic.

```
nirvighna@DESKTOP-VS57D2  x  +  v  -  □  ×

#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_OUTPUT 4096

void expand_variables(const char *input, char *output, size_t size)
{
    size_t i = 0;
    size_t j = 0;

    while (input[i] != '\0' && j < size - 1)
    {
        if (input[i] == '$')
        {
            char var_name[128];
            size_t k = 0;
            const char *value;

            i++;

            if (input[i] == '{')
            {
                i++;

                while (input[i] != '\0' &&
                    input[i] != '}' &&
                    k < sizeof(var_name) - 1)
                {
                    var_name[k++] = input[i++];
                }

                if (input[i] == '}')
                    i++;
            }
            else
            {
                while (input[i] != '\0' &&
                    input[i] != ' ' &&
                    input[i] != '\n')
                    i++;
            }

            value = getenv(var_name);

            if (value)
            {
                while (*value)
                    output[j++] = *value++;
            }
            else
            {
                output[j++] = '\0';
            }
        }
        else
        {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';
}

int main(int argc, char *argv[])
{
    if (argc < 2)
    {
        printf("Usage: %s <input>\n", argv[0]);
        return 1;
    }

    char *input = argv[1];
    char *output = malloc(MAX_OUTPUT);
    if (!output)
    {
        printf("Memory allocation failed\n");
        return 1;
    }

    expand_variables(input, output, MAX_OUTPUT);

    printf("Expanded: %s\n", output);

    free(output);
    return 0;
}

EOF return 0;f("Expanded: %s\n", output);sizeof(output));ame);
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ls -l bin/var_expand
-rwxr-xr-x 1 nirvighna nirvighna 20008 Sep 27 16:23 bin/var_expand
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/var_expand
Variable Expansion Shell
Type 'exit' to quit.
expand-shell> $USER
Expanded: nirvighna
expand-shell> |
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/var_expand
Variable Expansion Shell
Type 'exit' to quit.
expand-shell> $USER
Expanded: nirvighna
expand-shell> $ HOME
Expanded: $ HOME
expand-shell> expand-shell>
Expanded: expand-shell>
expand-shell> Expanded: /home/nirvighna
Expanded: Expanded: /home/nirvighna
expand-shell> Hello $USER, your home is $HOME
Expanded: Hello nirvighna, your home is /home/nirvighna
expand-shell> $THIS_VARIABLE_DOES_NOT_EXIST
Undefined variable: THIS_VARIABLE_DOES_NOT_EXIST
Expanded:
expand-shell> $THIS_VARIABLE_DOES_NOT_EXIST
Undefined variable: THIS_VARIABLE_DOES_NOT_EXIST
Expanded:
expand-shell> /home/$USER
Expanded: /home/nirvighna
expand-shell> |
```

TASK 2:

```
nirvighna@DESKTOP-VS57D2  x  +  v  -  □  ×

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>

#define MAX_INPUT 1024
#define MAX_ARGS 64

typedef int (*builtin_func)(char **args);

int builtin_cd(char **args)
{
    if (args[1] == NULL)
    {
        fprintf(stderr, "cd: missing argument\n");
        return 1;
    }

    if (chdir(args[1]) != 0)
    {
        perror("cd");
        return 1;
    }

    return 1;
}

int builtin_pwd(char **args)
{
    char cwd[1024];

    (void)args;

    if (getcwd(cwd, sizeof(cwd)) != NULL)
        printf("%s\n", cwd);
    else
        perror("pwd");

    return 1;
}

EOF return 0;hell exited.\n");mmand: %s\n", args[0]);L)[0]);
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ |

nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/builtin_shell
Built-in Dispatch Shell
Type 'help' for commands.
builtin-shell> |
```

```
nirvighna@DESKTOP-VS57D2I:~/OS-Week1$ ./bin/builtin_shell
```

```
Built-in Dispatch Shell
```

```
Type 'help' for commands.
```

```
builtin-shell> help
```

```
Built-in commands:
```

cd <directory>	Change directory
pwd	Show current directory
echo <text>	Display text
help	Show built-in commands
exit	Exit shell

```
builtin-shell> cd /tmp
```

```
builtin-shell> pwd
```

```
/tmp
```

```
builtin-shell> |
```